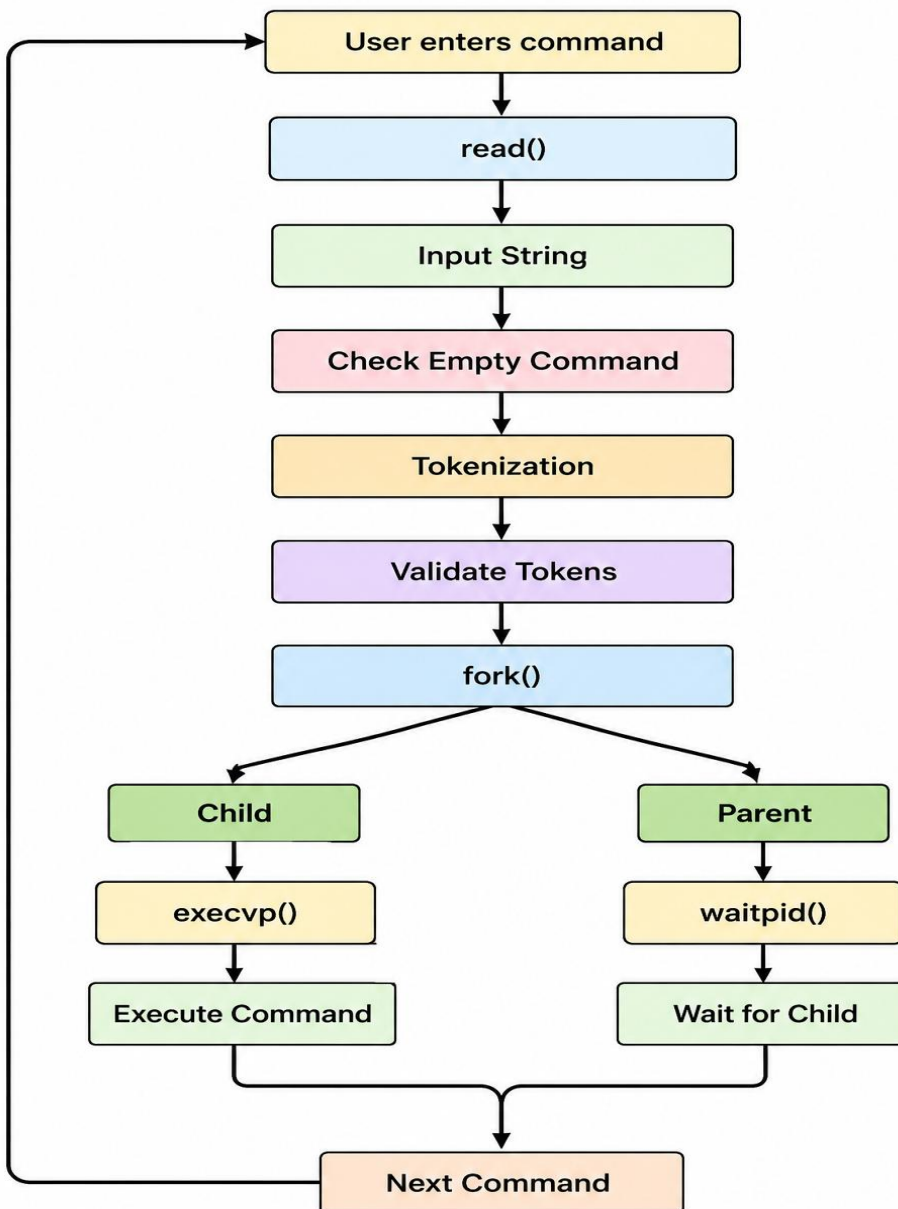


Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Flow Chart:





sindhuja@SINDHUJA: ~/skill_



```
GNU nano 7.2 skill4_task1.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_TOKENS 20

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int status;

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, MAX_INPUT, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command! Please enter a command.\n");
            continue;
        }

        int count = 0;
        char *token = strtok(input, " ");

        while (token != NULL)
        {
            tokens[count] = token;
            printf("Token %d: %s\n", count + 1, tokens[count]);
            count++;
            token = strtok(NULL, " ");
        }

        tokens[count] = NULL;

        pid_t pid = fork();

        if (pid == 0)
        {
            printf("Child Process PID: %d\n", getpid());

            execvp(tokens[0], tokens);

            printf("Invalid command!\n");
            exit(1);
        }
        else
        {
            printf("Parent Process PID: %d\n", getpid());

            waitpid(pid, &status, 0);

            printf("Command execution completed.\n\n");
        }
    }

    return 0;
}
```

```
sindhuja@SINDHUJA: ~/skill_ x + v
sindhuja@SINDHUJA:~/skill_week4$ nano skill4_task1.c

sindhuja@SINDHUJA:~/skill_week4$ gcc skill4_task1.c -o skill4_task1
sindhuja@SINDHUJA:~/skill_week4$ ls
skill4_task1 skill4_task1.c
sindhuja@SINDHUJA:~/skill_week4$ ./skill4_task1
myShell> ls
Token 1: ls
Parent Process PID: 562
Child Process PID: 563
skill4_task1 skill4_task1.c
Command execution completed.

myShell> pwd
Token 1: pwd
Parent Process PID: 562
Child Process PID: 564
/home/sindhuja/skill_week4
Command execution completed.

myShell> echo Hello Linux
Token 1: echo
Token 2: Hello
Token 3: Linux
Parent Process PID: 562
Child Process PID: 565
Hello Linux
Command execution completed.

myShell>
Empty command! Please enter a command.
myShell> abc
Token 1: abc
Child Process PID: 568
Parent Process PID: 562
Invalid command!
Command execution completed.

myShell> ^C
sindhuja@SINDHUJA:~/skill_week4$ ./skill4_task1
myShell> ls
Token 1: ls
Parent Process PID: 569
Child Process PID: 570
skill4_task1 skill4_task1.c
Command execution completed.

myShell> pwd
Token 1: pwd
Parent Process PID: 569
Child Process PID: 571
/home/sindhuja/skill_week4
Command execution completed.

myShell> echo Hello Linux
Token 1: echo
Token 2: Hello
Token 3: Linux
Parent Process PID: 569
Child Process PID: 578
Hello Linux
Command execution completed.

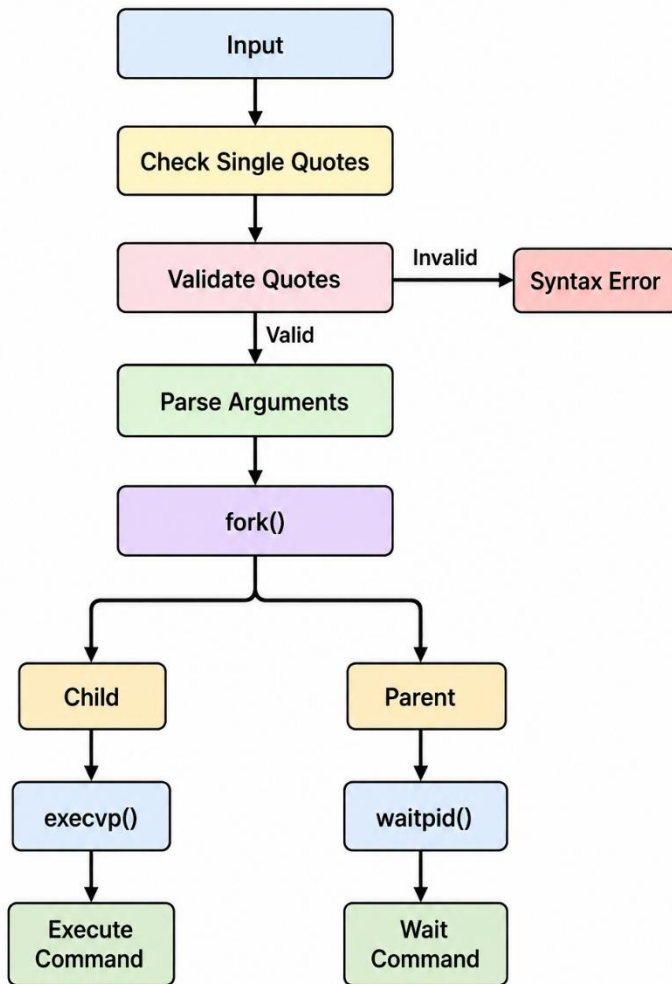
myShell>
Empty command! Please enter a command.
myShell> abc
Token 1: abc
Parent Process PID: 569
Child Process PID: 579
Invalid command!
Command execution completed.

myShell> ^C
sindhuja@SINDHUJA:~/skill_week4$ |
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:



Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Hello

Command execution completed.

Case2:

```
myShell> echo 'Hello World'
```

Syntax Error: Unmatched single quote.

```
GNU nano 7.2 skill4_task2.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, MAX_INPUT, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command!\n");
            continue;
        }

        int quoteCount = 0;
        for (int i = 0; input[i] != '\0'; i++)
        {
            if (input[i] == '\\')
                quoteCount++;
        }

        if (quoteCount % 2 != 0)
        {
            printf("Syntax Error: Unmatched single quote.\n\n");
            continue;
        }

        int count = 0;
        char *ptr = input;

        while (*ptr)
        {
            while (*ptr == ' ')
                ptr++;

            if (*ptr == '\0')
                break;

            if (*ptr == '\\')
            {
                ptr++;
                args[count++] = ptr;

                while (*ptr && *ptr != '\\')
                    ptr++;

                if (*ptr)

```

```
sindhuja@SINDHUJA: ~/skill_ × + ▾
GNU nano 7.2 skill4_task2.c *
ptr++;
if (*ptr == '\0')
    break;
if (*ptr == '\n')
{
    ptr++;
    args[count++] = ptr;
    while (*ptr && *ptr != '\n')
        ptr++;
    if (*ptr)
    {
        *ptr = '\0';
        ptr++;
    }
}
else
{
    args[count++] = ptr;
    while (*ptr && *ptr != ' ')
        ptr++;
    if (*ptr)
    {
        *ptr = '\0';
        ptr++;
    }
}
}
args[count] = NULL;
printf("Parsing Result:\n");
for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}
pid_t pid = fork();
if (pid == 0)
{
    printf("Child PID: %d\n", getpid());
    execvp(args[0], args);
    printf("Invalid command.\n");
    exit(1);
}
else
{
    waitpid(pid, NULL, 0);
    printf("Command execution completed.\n\n");
}
return 0;
}
⌕ Help ⌕ Write Out ⌕ Where Is ⌕ Cut ⌕ Execute ⌕ Location ⌕ Undo ⌕ Set Mark
⌕ Exit ⌕ Read File ⌕ Replace ⌕ Paste ⌕ Justify ⌕ Go To Line ⌕ Redo ⌕ Copy
```

```
myShell> ^C
sindhuja@SINDHUJA:~/skill_week4$ nano skill4_task2.c
sindhuja@SINDHUJA:~/skill_week4$ gcc skill4_task2.c -o skill4_task2
sindhuja@SINDHUJA:~/skill_week4$ ./skill4_task2
myShell> echo Hello
Parsing Result:
Argument 1: echo
Argument 2: Hello
Child PID: 601
Hello
Command execution completed.

myShell> echo 'Hello World'
Parsing Result:
Argument 1: echo
Argument 2: Hello World
Child PID: 604
Hello World
Command execution completed.

myShell> echo 'Hello World
Syntax Error: Unmatched single quote.

myShell> ^C
sindhuja@SINDHUJA:~/skill_week4$ echo Hello
Hello
sindhuja@SINDHUJA:~/skill_week4$ echo 'Hello World'
Hello World
sindhuja@SINDHUJA:~/skill_week4$ echo 'Hello World
> '
Hello World

sindhuja@SINDHUJA:~/skill_week4$ |
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Example Output:**Case1:**

myShell> echo "Hello \$USER"

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process

PID = 3022

Hello raghu

Child process completed.

Case2:

myShell> echo "Hello World

Syntax Error: Unmatched double quote.

sindhuja@SINDHUJA: ~/skill_

skill4_task3.c *

```
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, MAX_INPUT, stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Empty command!\n");
            continue;
        }

        int count = 0;
        char *ptr = input;

        while (*ptr)
        {
            while (*ptr == ' ')
                ptr++;

            if (*ptr == '\0')
                break;

            if (*ptr == '"')
            {
                ptr++;
                args[count++] = ptr;

                while (*ptr && *ptr != '"')
                    ptr++;

                if (*ptr)
                {
                    *ptr = '\0';
                    ptr++;
                }
            }
            else
            {
                args[count++] = ptr;

                while (*ptr && *ptr != ' ')
                    ptr++;

                if (*ptr)

```

^G Help

^X Exit

^O Write Out

^R Read File

^W Where Is

^L Replace

^K Cut

^U Paste

^T Execute

^J Justify

^C Location

^_ Go To Line

M-U Undo

M-E Redo

M-A Set Mark

M-G Copy

```
sindhuja@SINDHUJA: ~/skill_  × + ▾
GNU nano 7.2 skill4_task3.c
ptr++;
args[count++] = ptr;

while (*ptr && *ptr != '\0')
    ptr++;

if (*ptr)
{
    *ptr = '\0';
    ptr++;
}
}
else
{
    args[count++] = ptr;

    while (*ptr && *ptr != ' ')
        ptr++;

    if (*ptr)
    {
        *ptr = '\0';
        ptr++;
    }
}
}

args[count] = NULL;

for (int i = 0; i < count; i++)
{
    if (args[i][0] == '$')
    {
        char *value = getenv(args[i] + 1);
        if (value != NULL)
            args[i] = value;
    }
}

printf("Parsing Result:\n");
for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

pid_t pid = fork();

if (pid == 0)
{
    execvp(args[0], args);
    printf("Invalid command.\n");
    exit(1);
}
else
{
    waitpid(pid, NULL, 0);
    printf("Command execution completed.\n\n");
}
}

return 0;
}
```

```
sindhuja@SINDHUJA:~/skill_week4$ nano skill4_task3.c
sindhuja@SINDHUJA:~/skill_week4$ gcc skill4_task3.c -o skill4_task3
sindhuja@SINDHUJA:~/skill_week4$ ./skill4_task3
myShell> echo "Hello World"
Parsing Result:
Argument 1: echo
Argument 2: Hello World
Hello World
Command execution completed.

myShell> echo "$HOME"
Parsing Result:
Argument 1: echo
Argument 2: /home/sindhuja
/home/sindhuja
Command execution completed.

myShell> echo "$USPER"
Parsing Result:
Argument 1: echo
Argument 2: $USPER
$USPER
Command execution completed.

myShell> pwd
Parsing Result:
Argument 1: pwd
/home/sindhuja/skill_week4
Command execution completed.

myShell> abc
Parsing Result:
Argument 1: abc
Invalid command.
Command execution completed.

myShell> ^C
sindhuja@SINDHUJA:~/skill_week4$
```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
sindhuj@SINDHUJA: ~/skill_  × + ∨
GNU nano 7.2 skill4_task4.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1)
    {
        printf("Pipe creation failed!\n");
        return 1;
    }

    pid1 = fork();

    if (pid1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);

        char *cmd1[] = {"ls", NULL};
        execvp(cmd1[0], cmd1);

        exit(1);
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[1]);
        close(fd[0]);

        char *cmd2[] = {"wc", "-l", NULL};
        execvp(cmd2[0], cmd2);

        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline executed successfully.\n");

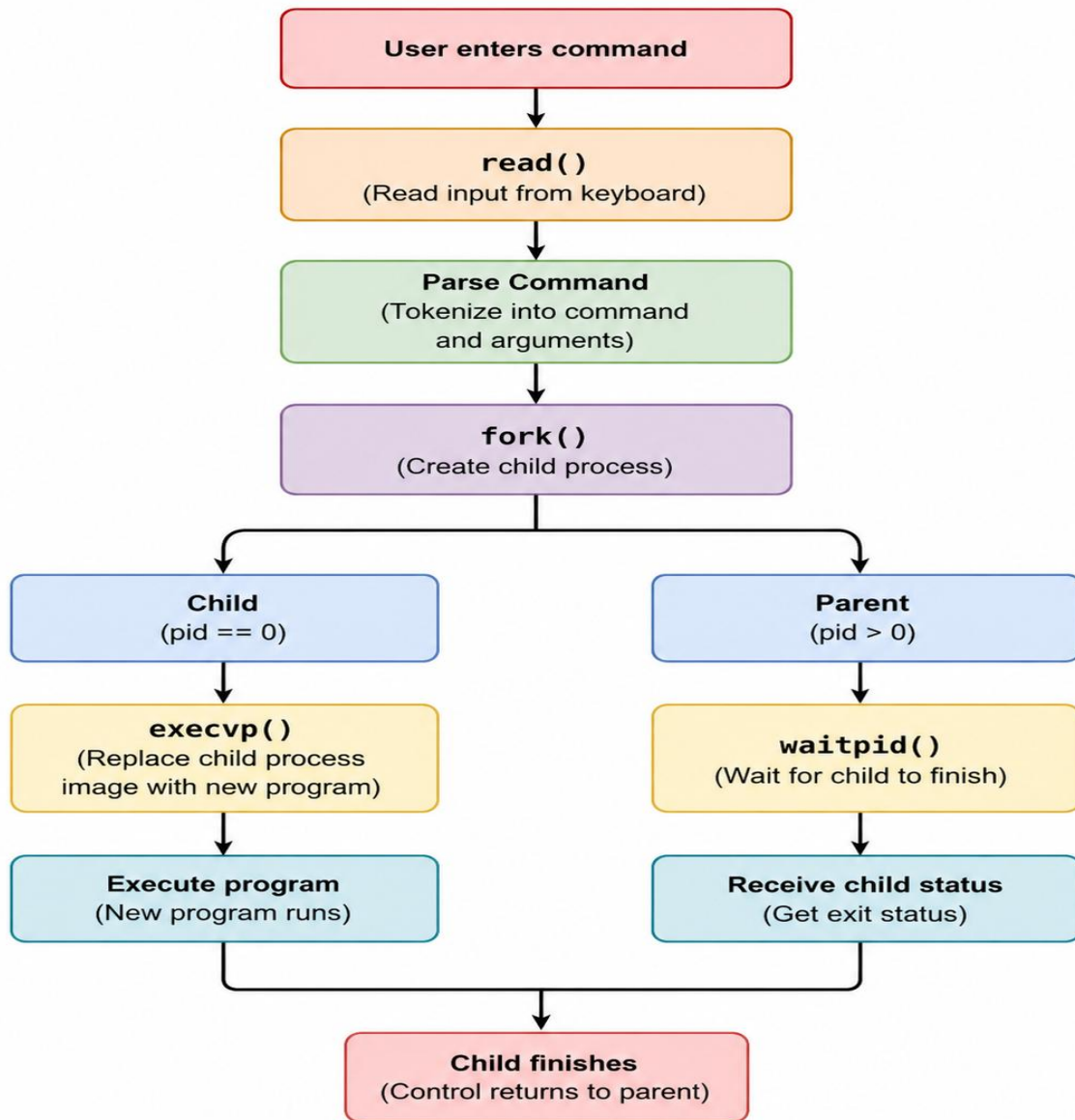
    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location ^U Undo ^-A Set Mark ^-6 Copy
^X Exit ^R Read File ^_ Replace ^J Paste ^_ Justify ^/ Go To Line ^-E Redo ^-6 Copy

```
myShell> ^C
sindhuj@SINDHUJA:~/skill_week4$ nano skill4_task4.c
sindhuj@SINDHUJA:~/skill_week4$ gcc skill4_task4.c -o skill4_task4
sindhuj@SINDHUJA:~/skill_week4$ ./skill4_task4
8
Pipeline executed successfully.
sindhuj@SINDHUJA:~/skill_week4$
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Flow Chart:



Sample Output:

Case 1:

myshell\$ ls

Parent Process PID: 1200

Created Child PID: 1201

Child Process PID: 1201

file1.txt

file2.txt

Child exited with status: 0

Case 2:

myshell\$ ls @l

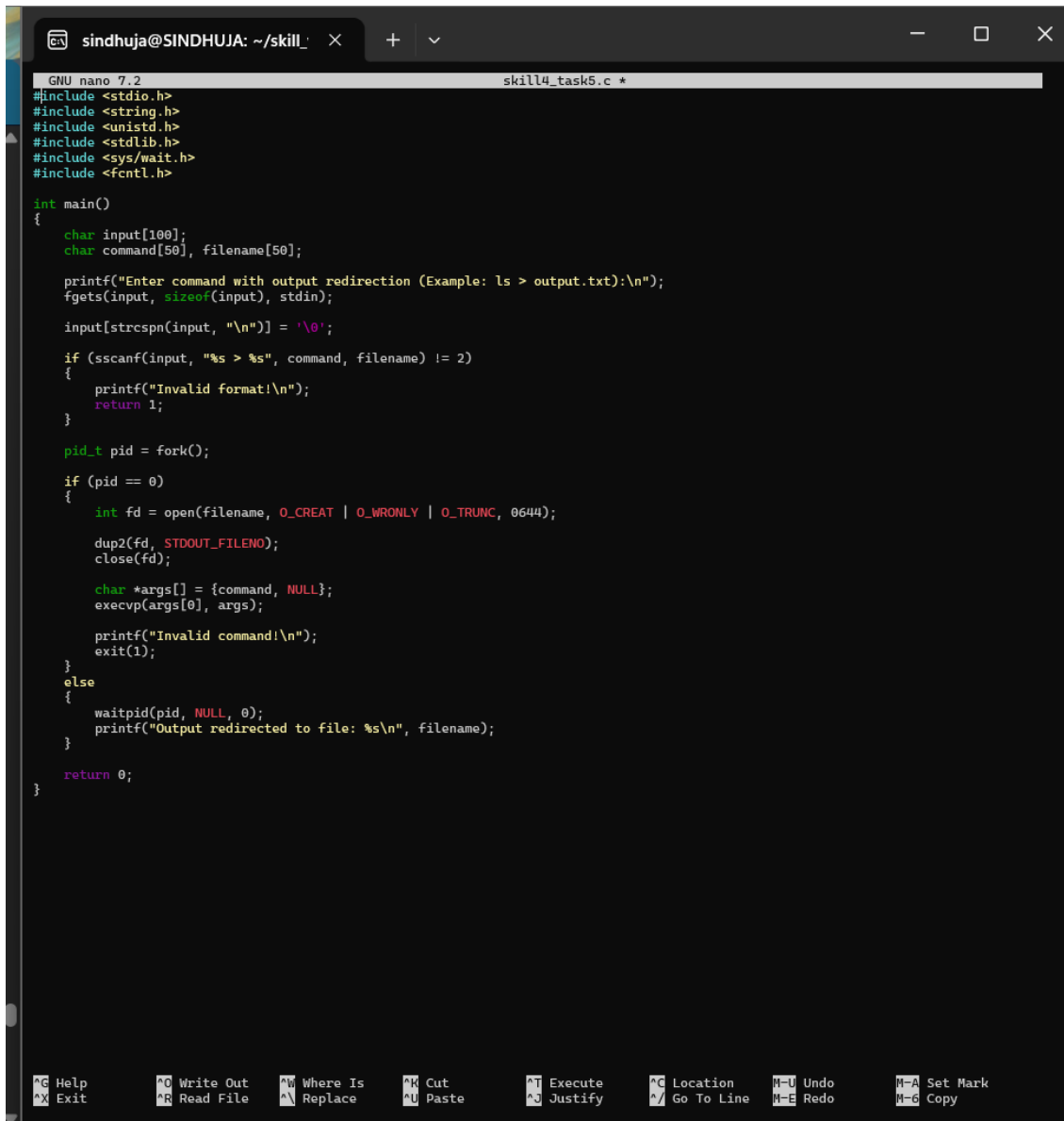
Parent Process PID: 82

Child Process PID: 83

Created Child PID: 83

ls: cannot access '@l': No such file or directory

Child exited with status: 2



```
GNU nano 7.2 skill4_task5.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <fcntl.h>

int main()
{
    char input[100];
    char command[50], filename[50];

    printf("Enter command with output redirection (Example: ls > output.txt):\n");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (sscanf(input, "%s > %s", command, filename) != 2)
    {
        printf("Invalid format!\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid == 0)
    {
        int fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0644);

        dup2(fd, STDOUT_FILENO);
        close(fd);

        char *args[] = {command, NULL};
        execvp(args[0], args);

        printf("Invalid command!\n");
        exit(1);
    }
    else
    {
        waitpid(pid, NULL, 0);
        printf("Output redirected to file: %s\n", filename);
    }

    return 0;
}
```

```
myShell> ^C
sindhuj@SINDHUJA:~/skill_week4$ nano skill4_task4.c
sindhuj@SINDHUJA:~/skill_week4$ gcc skill4_task4.c -o skill4_task4
sindhuj@SINDHUJA:~/skill_week4$ ./skill4_task4
8
Pipeline executed successfully.
sindhuj@SINDHUJA:~/skill_week4$ nano skill4_task5.c
sindhuj@SINDHUJA:~/skill_week4$ gcc skill4_task5.c -o skill4_task5
sindhuj@SINDHUJA:~/skill_week4$ ./skill4_task5
Enter command with output redirection (Example: ls > output.txt):
ls > output.txt
Output redirected to file: output.txt
sindhuj@SINDHUJA:~/skill_week4$ cat output.txt
output.txt
skill4_task1
skill4_task1.c
skill4_task2
skill4_task2.c
skill4_task3
skill4_task3.c
skill4_task4
skill4_task4.c
skill4_task5
skill4_task5.c
sindhuj@SINDHUJA:~/skill_week4$ ./skill4_task5
Enter command with output redirection (Example: ls > output.txt):
pwd > path.txt
Output redirected to file: path.txt
sindhuj@SINDHUJA:~/skill_week4$ cat path.txt
/home/sindhuj/skill_week4
sindhuj@SINDHUJA:~/skill_week4$ nano skill4_task5.c
sindhuj@SINDHUJA:~/skill_week4$ gcc skill4_task5.c -o skill4_task5
sindhuj@SINDHUJA:~/skill_week4$ ./skill4_task5
Enter command with output redirection (Example: ls > output.txt):
ls > output.txt
Output redirected to file: output.txt
sindhuj@SINDHUJA:~/skill_week4$ pwd > path.txt
sindhuj@SINDHUJA:~/skill_week4$ cat output.txt
output.txt
path.txt
skill4_task1
skill4_task1.c
skill4_task2
skill4_task2.c
skill4_task3
skill4_task3.c
skill4_task4
skill4_task4.c
skill4_task5
skill4_task5.c
sindhuj@SINDHUJA:~/skill_week4$ cat path.txt
/home/sindhuj/skill_week4
sindhuj@SINDHUJA:~/skill_week4$ |
```